

Paralelni Računarski Sistemi

Skripta:

Srećko Glibetić

Sadržaj

1. Koje vrste benchmark programa postoje?	4
2. Šta je efektivni paralelni algoritam? Koju osobinu poseduje?	4
3. Navesti mere performansi sistema.	5
4. Karakteristike CISC procesora?	6
5. Karakteristike RISC procesora?	6
6. Formati instrukcija kod RISC procesora.	7
7. U kojim fazama izvršenja se pristupa registarskom fajlu kod RISC procesora?	8
8. Koji hazardi se javljaju kod protočnih sistema? Ukratko ih objasniti.	8
9. Zakašnjeni LOAD?	8
10. Opistati tehniku premošćavanja za razrešavanje RAW hazarda.	9
11. Ako su I i J instrukcije u programu, pri čemu I prethodi J, objasniti kada nastupaju RAW, WAR i WAW hazardi	9
12. Kada se detektuju svi hazardi?	9
13. U kojoj fazi protočnog sistema se razrešava grananje?	10
14. Pbrojati hardverske tehnike za smanjenje kašnjenja uzrokovanog kontrolnim hazardima.	10
15. Kako se rešava problem prekida kod protočnog izvršenja FP instrukcija?	10
16. Faze izvršenja instrukcija kod Scoreboard-a	11
17. U kom stepenu protočnog sistema koji koristi Scoreboard dolazi do preuređenja instrukcija koje dovodi do izvršenja van redosleda?	11
18. U kojoj fazi izvršenja instrukcija se kod Scoreboard tehnike detektuju WAW hazardi, a u kojoj WAR hazardi?	11
19. Faze izvršenja instrukcija kod Tomasula.	12
20. Navesti razlike između Scoreboard i Tomasula.	13
21. Pbrojati tehnike za smanjenje kašnjenja uzrokovano kontrolnim hazardima (nabrojati tehnike predikcije).	13
22. U kojoj fazi se pristupa BPB-u?	14
23. U kojoj fazi se pristup BTB-u?	14
24. Opisati rad n-to bitnog prediktora za predikciju grananja.	14
25. Odrediti srednje gubitke u clock-u za naredbu grananja ako se koristi BTB, usvajajući da je: pouzdanost predviđanja 80%, stopa pogotka u BTB 90%, grananja se dešavaju u 70% slučajeva. Broj izgubljenih clock ciklusa u slučaju pogrešne predikcije i u slučaju da se naredba ne nađe u BTB iznosi 2 clock ciklusa.	14
26. Osnovne komponente vektorskih računara	15
27. Po kojoj petlji u gnezdu petlji se vrši vektorizacija?	15
28. Koji hazardi mogu sprečiti vektorizaciju petlji?	15
29. Koje vrste zavisnosti sprečavaju vektorizaciju?	16
30. Kako se određuju vektori i matrice zavisnosti po podacima?	16
31. Odrediti matricu zavisnosti po podacima za sledeće gnezdo petlji.	16
32. Koje osobine moraju da zadovolje matrice transformacije indeksnih promenljivih?	17
33. Data je matrica zavisnosti po podacima i matrica transformacije; Da li je dozvoljeno izvršiti transformaciju petlji primenom transformacije T? Zašto?	17
34. Koje se elementarne transformacije mogu obavljati nad indeksnim promenljivim u petljama?	17
35. Koliko vremena će biti potrebno da se napuni vektor:	18
36. Ako je latentnost memorije 8 clk ciklusa, a vektorski korak 10, koliko je najmanje memorijskih banaka potrebno da bi se elementima vektora pristupalo bez konflikata ako broj banaka mora biti stepen dvojke?	18
37. Kako se može obaviti maskiranje procesnih elemenata u SIMD?	19

38. Nacrtati strukturu procesnog elementa kod procesiranja sa distribuiranom memorijom	19
39. U SIMD sistemu postoji 16 procesnih elemenata. PE-ovima sa adresama 0, 1, 3, 9, 11, 15 treba dozvoliti rad, a ostalima zabraniti. Kako izgledaju maske kojima se ovo postiže ako se koristi direktno maskiranje, a kako ako se koristi adresno maskiranje?	20
40. Kojim sprežnim funkcijama su definisani mešanje-zamena, kub i rešetka?.....	20
41. Kako se određuje zaglavlje poruke koje definiše put kroz sprežnu mrežu generalizovani kub u slučaju da se vrši emisija?	21
42. U sistemu postoji 16 procesora. Za povezivanje se koristi višestepena sprežna mreža generalizovani kub. Procesor P1 treba da pošalje poruku procesorima P4, P5, P6, P7, P12, P13, P14 i P15. Da li je emisija moguća? Zašto? Kako izgleda zaglavlje koje se dodaje poruci?	21
43. Navesti karakteristike multiprocesora i multiračunara.	22
44. Navesti politike dodele magistrale kod multiprocesorskih sistema.	23
45. Navesti politike oslobađanja magistrale.	23
46. Objasniti arbitražu na magistrali koja se zasniva na deljivim zahtevima i lančanom zahvatanju. ..	24
47. Objasniti distribuiranu arbitražu na magistrali.....	24
48. Tehnike za postizanje keš koherencije.	25
49. Koja stanja keš bloka postoje kod write-once protokola.	25
50. Opisati promenu stanja keš bloka u slučaju da se koristi Write-once protokol i da nastupi promašaj pri upisu.....	26
51. Koja stanja keš bloka postoje kod Firefly protokola?.....	26
52. Opisati promenu stanja keš bloka u slučaju da se koristi Firefly protokol i da nastupi promašaj pri upisu	26
53. Multiprocesorski sistem se sastoji od tri procesora. Svaki procesor ima svoju keš memoriju. Prikazati kako se vrši promena stanja keš blokova i glavne memorije ako se za održavanje keš koherencije koristi: Write-once protokol i Firefly protokol .Procesori pristupaju deljivoj promenljivoj X. Akcije procesora navedene su u prvoj koloni tabele. Popuniti ostale ćelije tabele.	26
54. Šta su direktorijumske šeme?.....	27
55. Kako se održava konzistentnost podataka kod direktorijumskih šema?.....	27
56. Protokoli sa potpuno preslikanim adresarima (full mapped directories).	27
57. Protokoli sa ograničenim adresarima (limited directories).	27
58. Objasniti kako izgleda formiranje direktorijumskog lanca ako se koristi direktorijumska šema sa ulančanim adresarima.	27
59. Koje su operacije dozvoljene nad semaforima? Dati opis operacija u pseudo kodu	28
60. Navesti razlike između semafora i monitora kao sredstava sinhronizacije i komunikacije procesa.....	28
61. Čemu služe podaci tipa CONDITION kod monitora, i koje se operacije mogu izvršavati nad podacima tog tipa?	28

1. Koje vrste benchmark programa postoje?

Benchmark je skup reprezentativnih programa za ocenu performansi računara koji daju relativnu sliku o performansama sistema. Vrste benchmark programa su:

- ⑩ Realni programi – izvršavaju se na ciljnoj arhitekturi (teško se mere performanse)
- ⑩ Miks realnih programa – tipični za ciljanu aplikaciju ili opterećenje
- ⑩ Jezgra – ključni delovi realnih programa sa intenzivnim izračunavanjem (koriste se za testiranje specijalnih aspekata mašine)
- ⑩ Igrački – kratki programi koji generišu poznati rezultat.
- ⑩ Sintetički – mali, specijalno kreirani programi sa ciljem da izoluju pojedine aspekte performansi.

2. Šta je efektivni paralelni algoritam? Koju osobinu poseduje?

Efektivni paralelni algoritam je onaj algoritam koji sa porastom obima problema, Amdalova frakcija α (udeo vremena izvršenja sekvencijalnog dela programa u ukupnom vremenu izvršenja) teži nuli:

$$\lim_{m \rightarrow \infty} \alpha(m) = 0$$

m – obim problema.

Efektivni algoritmi imaju linearno ubrzanje:

$$\lim_{m \rightarrow \infty} S(n) = \lim_{m \rightarrow \infty} \frac{n}{1 + (n-1)\alpha(m)} = n$$

3. Navesti mere performansi sistema.

9 Ubrzanje:

$$S = \frac{\text{Vreme izvršenja programaa arhitekturi bez poboljšanja}}{\text{Vreme izvršenja programaa arhitekture sa izvedenim poboljšanjem}}$$

Za paralelni sistem sa n procesora:

$$S(n) = \frac{T(1)}{T(n)} = \frac{T_s + T_p}{T_s + \frac{T_p}{n} + T_0(n)}$$

gde je:

- 10 $T(1)$ – vreme izvršenja na jednoprocesorskom sistemu
- 10 $T(n)$ – vreme izvršenja na višeprocorskom sistemu
- 10 T_s – vreme izvršenja sekvencijalnog dela programa
- 10 T_p – vreme izvršenja paralelnog dela programa
- 10 $T_0(n)$ – vreme potrebno za sinhronizaciju i komunikaciju procesa za n - procesore sisteme.

9 Efikasnost:

Definiše se kao odnos ubrzanja i broja procesora:

$$E(n) = \frac{S(n)}{n}, 0 \leq E \leq 1$$

Daje ocenu srednjoj iskorišćenosti procesora u sistemu, kada su svi procesori angažovani na rešavanju istog problema. Ako se zanemare U/I aktivnosti, efikasnost jednoprocesorskog sistema je 1 (100%).

9 Ubrzanje i efikasnost ne treba posmatrati odvojeno:

- 10 sa porastom broja procesora ubrzanje raste (do određene granice)
- 10 sa porastom broja procesora efikasnost opada
- 10 $S(n) * E(n)$ je najbolja ocena

4. Karakteristike CISC procesora?

Jedan od pravaca razvoja procesora je u povećanju kompleksnosti CPU-a:

- Povećanje broja instrukcija (120 – 350)
- Povećanje kompleksnosti instrukcija
- Povećanje broja mogućih načina adresiranja
- Veliki broj specijalizovanih, a mali broj registara opšte namene
- Instrukcije su različite dužine
- Mikroprogramsko upravljanje

Sve ovo je uvedeno zbog poboljšanja performansi i uprošćavanje pisanja kompilatora.

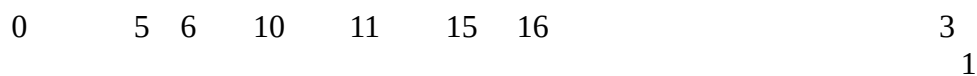
5. Karakteristike RISC procesora?

RISC procesori imaju suprotan prilaz problemu od CISC procesora. Ideja je da se instrukcije koje se često koriste implementiraju hardverski, dok instrukcije koje se koriste retko implementiraju softverski.

- Relativno mali broj instrukcija (<100)
- Sve instrukcije su iste dužine (32 b)
- Mali broj načina adresiranja
- Pristup memoriji se obavlja preko LOAD i STORE instrukcija
- Veliki broj registara opšte namene
- Najveći broj instrukcija je tipa registar-u-registar
- Hardversko, a ne mikroprogramsko upravljanje
- Jako izražena protočnost u obradi instrukcija
- Odvojeni keš za instrukcije i podatke
- Moćni kompilatori

6. Formati instrukcija kod RISC procesora.

9 I – format (load, store, branch, ALU – operacija sa neposrednim operandom)



OP	Rs1	Rd	Immed
----	-----	----	-------

- OP – kod operacije
- Rs1 – izvorni operand za ALU
 - bazni registar za generisanje memorijske adrese kod load i store
 - registar čiji se sadržaj testira za slučaj branch instrukcije
- Rd – polje odredišnog registra za slučaj ALU operacije i LOAD
 - polje registra čiji se sadržaj pamti za slučaj STORE
 - Rd = 0 za branch

9 R – format (ALU – operacija tipa registar-u-registar)



OP	Rs1	Rs2	Rd	funkcija
----	-----	-----	----	----------

- OP + funkcija – kod operacije
- Rs1 – prvi izvorni operand za ALU
- Rs2 – drugi izvorni operand za ALU
- Rd – polje odredišnog registra

9 J – format (instrukcija skoka – jump)



OP	offset (pomeraĵ)
----	------------------

- OP – kod operacije
- offset – pomeraj koji se dodaje PC (brojač instrukcija – program counter)

7. U kojim fazama izvršenja se pristupa registarskom fajlu kod RISC procesora?

Registarski fajl se koristi u dva stepena:

- Za čitanje u ID – fazi.
- Za upis u WB – fazi.

To znači da je u svakom klok ciklusu potrebno obaviti 2 čitanja i jedan upis.

8. Koji hazardi se javljaju kod protočnih sistema? Ukratko ih objasniti.

Hazardi su situacije koje onemogućuju izvršenje instrukcije u predviđenom klok ciklusu, što dovodi do redukcije idealnih performansi protočnih sistema. Postoje 3 vrste hazarda:

- Strukturni hazardi – nastaju zbog jednovremenog zahteva za korišćenjem istog hardverskog resursa.
- Hazardi po podacima – nastaju zato što je redosled pristupa operandima izmenjen uvođenjem protočnosti u odnosu na sekvencijalno izvršenje programa.
- Kontrolni hazardi – nastaju zbog instrukcija koje mogu promeniti sadržaj PC-a (branch, jump, call, return) kao posledica neblagovremenog izračunavanja adrese instrukcije koja će biti izvršena posle instrukcije grananja. Uzrokuju veći gubitak performansi nego hazardi po podacima.

9. Zakašnjeni LOAD?

Kod LOAD instrukcije nije moguće iskoristiti premošćavanje da bi se izbeglo zaustavljanje protočnog sistema.

Umesto zaustavljanja protočnog sistema kompajler može preurediti kod i izbeći zaustavljanje, preuređivanjem koda u fazi kompilacije moguće je izbeći hazarde tipa RAW prouzrokovane LOAD instrukcijom. Kompajler će izbeći generisanje LOAD instrukcije iza koje odma sledi instrukcija koja kao izvorišni operand ima ono što je za LOAD odredišni.

10. Opistati tehniku premošćavanja za razrešavanje RAW hazarda.

Posmatrajmo ovaj deo koda. Ovde imamo hazard tipa RAW, sve instrukcije posle ADD koriste registar R1 kao izvorišni registar, dok instrukcija ADD taj registar koristi kao odredišni, tj. Instrukcije SUB, AND, OR, XOR koriste rezultat operacije ADD. U protočnim sistemima instrukcija SUB počinje sa izvršavanjem odmah posle instrukcije ADD, ali pošto instrukcija ADD nije upisala rezultat u registar R1, SUB ne može nastaviti sa izvršavanjem dokle god se rezultat instrukcije ADD ne upiše u registar R1. To dovodi do zaustavljanja protočnog sistema, zbog detekcije hazarda.

```
ADD R1, R2, R3
SUB  R4, R1, R5
AND  R6, R1, R7O
OR   R8, R1, R9
XOR  R10, R1, R11
```

Zaustavljanje protočnog sistema se može izbeći jednostavnom tehnikom – pribavljanje unapred (forwardin, bypassing). Rezultat ALU operacije se uvek vraća na ALU ulazne lečeve. Ako hardver za detekciju zavisnosti otkrije da dve susedne instrukcije dele izvor i odredište, upravljačka logika selektuje premošćeni rezultat, ne vrednost učitane iz registarskog fajla. Premošćeni rezultat potrebno je proslediti i instrukciji AND. WB faza ADD i ID faza OR se poklapaju, problem za ovu istrukciju se rešava tako što se upis u registarski fajl vrši u prvoj polovini klok ciklusa, a čitanje u drugoj polovini.

11. Ako su I i J instrukcije u programu, pri čemu I prethodi J, objasniti kada nastupaju RAW, WAR i WAW hazardi

- RAW (Read After Write) – nastupa kada instrukcija J pokuša da pročita operand pre nego što je instrukcija I obavila upis (najčešći tip hazarda).
- WAR (Write After Read) – nastupa kada instrukcija J pokuša da upiše novu vrednost pre nego što je instrukcija I obavila čitanje.
- WAW (Write After Write) – nastupa kada instrukcija J pokuša da upiše novu vrednost pre instrukcije I.
- RAR – nije hazard

WAR i WAW su hazardi imenovanja, dok je RAW pravi hazard.

12. Kada se detektuju svi hazardi?

Svi hazardi se detektuju u ID fazi, ako hazard postoji protočni sistem se zaustavlja.

13. U kojoj fazi protočnog sistema se razrešava grananje?

Uslov grananja se razrešava u EXE fazi, a okončanje grananja u MEM fazi.

14. Pobrojati hardverske tehnike za smanjenje kašnjenja uzrokovanog kontrolnim hazardima.

- Ustanoviti što pre da li dolazi do grananja, za slučaj grananja, odrediti što pre sadržaj PC-a. Ako grananje vrši samo testiranje na nulu (BEQZ ili BNEZ) tada se testiranje može obaviti i u ID fazi kada se pristupa polju registara (RF). Novi sadržaj PC-a je moguće odrediti i na kraju ID faze, zahteva se novi ALU sabirač, jer je ALU zauzet u EXE fazi (ove dve faze se preklapaju).
- Predvideti da se grananje neće obaviti i u tom slučaju se stanje mašine neće promeniti dok ishod grananja ne bude poznat. Izvršavanje se nastavlja sa sledećom instrukcijom kao da nije u pitanju naredba grananja. Protočni sistem se zaustavlja ako se grananje obavi i predhodno pribavljena instrukcija se briše.
- Predvideti da se grananje uvek obavlja i početi pribavljnje instrukcije sa ciljne adrese grananja.
- Zakašnjeno grananje – instrukcija koja sledi iza naredbe grananja se izvršava bez obzira da li ima ili nema grananja. Uglavnom jedna instrukcija ide u delay slot. Postoje 3 mogućnosti za izbor te instrukcije:
 - ⑨ Nezavisna instrukcija koja se nalazi pre naredbe grananja.
 - ⑨ Instrukcija sa ciljne adrese grananja.
 - ⑨ Jedna od instrukcija koje se nalaze iza naredbe grananja.

15. Kako se rešava problem prekida kod protočnog izvršenja FP instrukcija?

- Baferovanje rezultata operacije dok se ne izvrše sve operacije koje su ranije počele sa izvršavanjem. Potreban je veliki baferski prostor za pamćenje rezultata, zahteva se i dodatni hardver za premošćavanje za rezultate koji se nalaze u redu čekanja, a nisu upisani u RF.
- Čuva se trag originalnih vrednosti registara svih instrukcija koje se nalaze u protočnom sistemu, tako da u slučaju prekida ima dovoljno informacija za restartovanje instrukcija koje se nalaze iza instrukcije koja je izazvala prekid.
- *Future file* – pamti novije vrednosti registara; kada se ranije izdata instrukcija okonča registarski fajl se ažurira iz *future file-a*. U slučaju prekida glavni registarski fajl ima originalne vrednosti operanada.

16. Faze izvršenja instrukcija kod Scoreboard-a

- **Issue** – Ako je funkcionalna jedinica slobodna i nijedna druga instrukcija nema isti odredišni registar scoreboard propušta instrukciju u funkcionalnu jedinicu i ažurira internu strukturu podataka. Ovde se detektuju strukturni i WAW hazardi, ako jedan od ova dva hazarda postoje onda se izdavanje novih instrukcija zaustavlja dok se on ne razreši.
- ⑨ Kada se Issue stepen zaustavi, bffer između IF i Issue stepena će se napuniti i prestaće pribavljanje novih instrukcija.
- ⑨ Instrukcije kroz Issue stepen prolaze po redosledu pribavljanja.
- **Read operands** – Scoreboard nadgleda raspoloživost izvornih operandata. Operand je raspoloživ ako ni jedna predhodna instrukcija neće da ga generiše (upiše rezultat).
- ⑨ Kada su operandi dostupni scoreboard kaže funkcionalnoj jedinici da pročita operande iz registarskog fajla i otpočne izvršenje.
- ⑨ Scoreboard rešava RAW hazarde u ovom koraku i instrukcija može biti poslata u izvršenje van redosleda.
- **Execution** – FU otpočinje izvršenje nakon pribavljanja operandata, kada je rezultat spreman FU obaveštava scoreboard.
- **Write result** – Kada je Scoreboard obavešten da je FU okončala sa izvršenjem, proverava WAR hazarde i zaustavlja instrukciju ako je potrebno. Ako WAR ne postoji ili se obriše Scoreboard kaže FU da upiše rezultat.

17. U kom stepenu protočnog sistema koji koristi Scoreboard dolazi do preuređenja instrukcija koje dovodi do izvršenja van redosleda?

U Read Operands fazi, koja se obrađuje u okviru ID stepena protočnog sistema (ID faza se deli na Issue i Read operands).

18. U kojoj fazi izvršenja instrukcija se kod Scoreboard tehnike detektuju WAW hazardi, a u kojoj WAR hazardi?

WAW hazardi se detektuju u **Issue fazi**, a WAR hazardi u **Write result** fazi.

19. *Faze izvršenja instrukcija kod Tomasula.*

- **Issue** – instrukcija se pribavlja iz reda čekanja.
- ⑨ Ako je u pitanju FP operacija, instrukcija se izdaje ako postoji slobodna rezervaciona stanica i šalju se operandi ako su u registrima.
- ⑨ Ako je u pitanju LOAD ili STORE instrukcija, ona se izdaje ako ima slobodan load ili store bufer respektivno.
- ⑨ Ako nema slobodnih rezervacionih stanica ili load/store bufera onda postoji strukturni hazard i instrukcija se zaustavlja.
- ⑨ U ovom koraku se vrši i preimenovanje registara.

- **Execution** – Ako neki operand nije dostupan nadgleda se CDB (common data bus).
- ⑨ Kada operand bude dostupan smešta se u odgovarajuću RS.
- ⑨ Kada su oba operanda dostupna, izvršava se odgovarajuća FP.
- ⑨ Vrši se provera RAW hazarda.
- ⑨ LOAD i STORE zahtevaju dvostepeno izvršenje.
- U prvom koraku se izračunava efektivna adresa, a zatim se pamti u LOAD/STORE bufer.
- Pribavljanje u LOAD bufer se obavlja čim je memorijska jedinica slobodna.
- Upisi u STORE bufere čekaju na podatak pre nego što se obavi upis u memoriju.

- **Write result** – Kada je rezultat dostupan upisuje se na CDB, a odatle u registre i bilo koju FU koja čeka taj rezultat.
- ⑨ Označiti da je FU slobodna.

20. *Navesti razlike između Scoreboard i Tomasula.*

- ⑩ Upravljanje distribuirano po rezervacionim stanicama kod Tomasulovog algoritma, naspram centralizovanog upravljanja kod Scoreboard-a.
- ⑩ FP registri opšte namene u instrukcijama su zamenjeni vrednostima (ako su dostupne), ili oznakama rezervacionih stanica (RS) koje će generisati vrednosti.
- ⑨ Ovaj proces se zove preimenovanje registara.
- ⑨ Izbegavaju se WAR i WAW hazardi.
- ⑨ Omogućava hardversko odmotavanje petlji.
- ⑨ Ima više RS nego registara opšte namene, što omogućava optimizacije koje kompajler ne može da obavi, pa broj registara opšte namene ne predstavlja usko grlo.
- ⑩ Resultati se prosleđuju FU iz rezervacionih stanica preko CDB-a, a ne preko registara, preko kojih se rezultati distribuiraju svim FU.
- ⑩ LOAD i STORE se tretiraju kao FU sa svojim RS.

21. *Pobrojati tehnike za smanjenje kašnjenja uzrokovano kontrolnim hazardima (nabrojati tehnike predikcije).*

Učestalost naredbi grananja u programima zahteva da se smanje potencijalni zastoji uzrokovani kontrolnim hazardima. Postoje dva načina smanjenja uticaja kontrolnih hazarda na izvršavanje programa:

- Statičke šeme
- ⑨ Zakašnjeno grananje
- ⑨ Odmotavanje petlje
- Dinamička predikcija grananja – razlikuje se od statičke jer koristi ponašanje grananja u toku izvršenja programa da predvidi njegov ishod.
- ⑨ Jedno-nivovske ili bimodalne – koriste *branch history table (BHT)* ili *branch prediction buffer (BPB)*, tabelu koja sadrži jedno ili dvo-bitne prediktore koje se adresiraju nižim bitovima adrese grananja.
- ⑨ Dvo-nivovski ili korelacioni prediktori.
- ⑨ Baferi ciljne adrese grananja *branch target buffer (BTB)* – sadrže adrese naredbi grananja koje su se ostvarile zajedno sa adresom ciljne adrese grananja.
- ⑨ Hibridni prediktori – koriste kombinaciju dve ili više šema (obično dve) za predikciju grananja.

22. *U kojoj fazi se pristupa BPB-u?*

BPB-u (*branch prediction buffer*) se pristupa u ID fazi, kad se obavi dekodiranje.

23. U kojoj fazi se pristup BTB-u?

BTB-u (*branch target buffer*) se pristupa u toku IF faze.

24. Opisati rad n-to bitnog prediktora za predikciju grananja.

N-to bitni prediktor koristi zasićeni brojač za predikciju grananja. Vrednosti koje brojač može uzeti su u opsegu od $0 - [2^n - 1]$. Ako je vrednos brojača veća od 2^{n-1} predviđanje je da će se grananje obaviti, u suprotnom, da se ne obavlja. Brojač se inkrementira svaki put kad se grananje obavi, a dekrementira ako se ne obavi. Kada je vrednost brojača 0 nema dekrementiranja (brojač ne može da ima negativne vrednosti), a kada brojač ima vrednost $2^n - 1$ onda se više ne inkrementira (to je maksimalna vrednost brojača).

Proučavanja su pokazala da se dvo bitni prediktori ponašaju skoro isto kao i n-to bitni prediktori, pa se u većini sistema koriste 2-bitni prediktori.

25. Odrediti srednje gubitke u clock-u za naredbu grananja ako se koristi BTB, usvajajući da je: pouzdanost predviđanja 80%, stopa pogotka u BTB 90%, grananja se dešavaju u 70% slučajeva. Broj izgubljenih clock ciklusa u slučaju pogrešne predikcije i u slučaju da se naredba ne nađe u BTB iznosi 2 clock ciklusa.

gubici =

$$\begin{aligned} & \text{stopa_pogotka_u_BTB} * \text{procenat_pogrešnih_predikcija} * 2 + \\ & + (1 - \text{stopa_pogotka_u_BTB}) * \text{procenat_obavljenih_grananja} * 2 = \\ & = 0.9 * 0.2 * 2 + 0.1 * 0.7 * 2 = 0.5 \text{ clk} \end{aligned}$$

Gubici zbog naredbi grananja su svedeni na 0.5 clk.

26. *Osnovne komponente vektorskih računara*

- Memorija

- ⑨ Organizacija memorije direktno utiče na performanse sistema.
- ⑨ Ima funkciju da pamti program i podatke u skalarnom i vektorskom obliku
- ⑨ Podeljena je niz memorijskih banaka kojima se može jednovremeno pristupati.
- ⑨ Koristi se *low order interleaving* za adresiranje memorijskih banaka – podaci koji su smešteni na susednim adresama su smešteni u različite banke.
- ⑨ Prema tome odakle se pribavljaju operandi, vektorski računari se dele na:
 - vektorsko-registarske, gde sve vektorske instrukcije, izuzev LOAD i STORE, pribavljaju operande iz vektorskih registara i smeštaju rezultat u vektorske registre.
 - memorijsko-memorijske, gde se svi operandi pribavljaju i smeštaju u memoriju.

- Skalarni podsistem

- ⑨ Namenjen izvršenju skalarnih instrukcija.
- ⑨ Ne razlikuju se od protočne organizacije procesora.
- ⑨ Skalarni registri i protočne funkcionalne jedinice.

- Vektorski podsistem

- ⑨ Sadrži protočno organizovane funkcionalne jedinice za izvršavanje vektorskih instrukcija.
- ⑨ Veliki broj vektorskih registara.

27. *Po kojoj petlji u gnjezdu petlji se vrši vektorizacija?*

Kod vektorizacije gnjezda petlji, vektorizacija se vrši po najdublje ugnježdenoj petlji.

28. *Koji hazardi mogu sprečiti vektorizaciju petlji?*

RAW hazardi.

29. *Koje vrste zavisnosti sprečavaju vektorizaciju?*

Loop-carry zavisnosti, koje su posledica korišćenja vrednosti koje izračunate u predhodnim iteracijama petlje. Ove zavisnosti se elimišu preuređenjem naredbi unutar petlje ili razbijanjem petlje.

Loop-carry zavisnost postoji ako važi sledeće; Postoje dva iterativna indeksa j i k ($j < k$), oba unutar granica petlje, tako da se u petlji pamti rezultat u element sa indeksom $a*j+b$, a zatim se pribavlja (čita) isti element kada je indeksiran sa $c*k+d$, tj. kada je $a*j+b = c*k+d$.

30. *Kako se određuju vektori i matrice zavisnosti po podacima?*

Ako je generisana promenljiva $X(f(i))$, gde je f celobrojna funkcija definisana nad indeksnim skupom i , a $X(g(i))$ korišćena promenljiva (g je opet celobrojna funkcija definisana nad indeksnim skupom i), vektor zavisnosti se izračunava kao:

$$d = f(i) - g(i)$$

Ako su $d_1, d_2, \dots, d_k$, svi vektori zavisnosti, tada se matrica zavisnosti po podacima dobija kao:

$$D = [d_1, d_2, \dots, d_k]$$

31. *Odrediti matricu zavisnosti po podacima za sledeće gnjezdo petlji.*

```
do 10 i=1, 5
  do 10 j=1, 10
    do 10 k=1, 20
      x(i, j, k)= x(i-1, j, k+1) + y(i, j, k)
      y(i, j, k+1)= y(i, j-1, k-1) * x(i, j, k)
    10 continue
```

Uočimo prvo sve parove parove generisanih – korišćenih promenljivih:

$x(i, j, k)$ i $x(i-1, j, k+1)$

$x(i, j, k)$ i $x(i, j, k)$

$y(i, j, k+1)$ i $y(i, j, k)$

$y(i, j, k+1)$ i $y(i, j-1, k-1)$

Odgovarajući vektori zavisnosti su:

$$d_1 = (i, j, k)^T - (i-1, j, k+1)^T = [1, 0, -1]^T$$

$$d_1 = (i, j, k)^T - (i, j, k)^T = [0, 0, 0]^T$$

$$d_1 = (i, j, k+1)^T - (i, j, k)^T = [0, 0, 1]^T$$

$$d_1 = (i, j, k+1)^T - (i, j-1, k-1)^T = [0, 1, 2]^T$$

Matrica zavisnosti je:

$$D = [d_1, d_2, d_3, d_4] = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ -1 & 0 & 1 & 2 \end{bmatrix}$$

32. *Koje osobine moraju da zadovolje matrice transformacije indeksnih promenljivih?*

- To su kvadratne matrice, što znači da vrše preslikavanje n -dimenzionalnog indeksnog prostora u n -dimenzionalni indeksni prostor.
 - To su celobrojne matrice.
 - $|\det T| = 1$
- Matrica transformacije T ne sme da menja znak vektora zavisnosti.

**33. Data je matrica zavisnosti po podacima i matrica transformacije;
Da li je dozvoljeno izvršiti transformaciju petlji primenom transformacije T? Zašto?**

$$D = [d_1 \quad d_2 \quad d_3] = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 2 \end{bmatrix}$$

matrica zavisnosti

$$T = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}$$

matrica transformacije

rešenje:

$$\hat{D} = T \cdot D = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix} \cdot \begin{bmatrix} 1 & 1 & 0 \\ 0 & 0 & 1 \\ -1 & 1 & 2 \end{bmatrix} = \begin{bmatrix} -1 & 1 & 2 \\ 0 & 0 & 1 \\ 1 & 1 & 0 \end{bmatrix}$$

Nije dozvoljeno, zato što T menja znak vektoru zavisnosti d_1 . Ako se promeni znak prvog ne nultog elementa u koloni onda se menja znak vektora zavisnosti.

34. Koje se elementarne transformacije mogu obavljati nad indeksnim promenljivim u petljama?

- Permutacija – omogućava zamenu mesta dvema petljama.
- Obrtanje redosleda – Omogućava promenu redosleda izračunavanja po određenoj indeksnoj promenljivoj.
- Krivljenje – Ovom transformacijom se obavlja krivljenje (*skewing*) jednog iterativnog indeksa u odnosu na drugi za faktor f.

35. Koliko vremena će biti potrebno da se napuni vektor:

Imamo 16 memorijskih banaka sa latentnošću od 12 clk ciklusa. Koliko vremena će biti potrebno da se napuni 64-elementni vektor, ako se elementi koji se pribavljaju nalaze na međusobnom rastojanju:

⑩ 1 (tj. vektorski korak je 1),

⑩ 32 (tj. vektorski korak je 32)?

a) Pošto je broj memorijskih banaka (16) veći od latentnosti memorijskog sistema (12), za korak 1 imaćemo da je vreme da se pribave 64 elemnta:

$$12 + 64 = 76 \text{ klok ciklusa, ili } 1.2 \text{ clk/elementu}$$

b) $\frac{NZS(32,16)}{32} = \frac{32}{32} = 1 < 16$ latentnost je 12 clk ciklusa po elementu tj. za ceo vektor:

$$12 * 64 = 768 \text{ clk}$$

36. Ako je latentnost memorije 8 clk ciklusa, a vektorski korak 10, koliko je najmanje memorijskih banaka potrebno da bi se elementima vektora pristupalo bez konflikata ako broj banaka mora biti stepen dvojke?

$$\frac{NZS(k, \text{broj}_{\text{banaka}})}{k} > \text{latentnost}$$

$k=10$ – vektorski korak, latentnost = 8,

$$\frac{NZS(10, 2^x)}{10} > 8 \Rightarrow NZS(10, 2^x) > 80$$

10,16	2	
5,8	2	
5,4	2	
5,2	2	$\Rightarrow x = 5 \Rightarrow \text{broj}_{\text{banaka}} = 2^5 = 32$
5,1	5	
1,1	=80	

37. Kako se može obaviti maskiranje procesnih elemenata u SIMD?

Svaka maska deli skup PE-ova na podskup aktivnih i podskup pasivnih. Može se obaviti:

- Statički

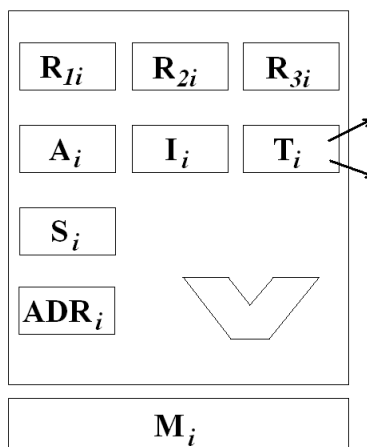
- Direktno (maskira se proizvoljan podskup PE-ova). Sadržaj registra maske određuje kompilator nakon prevođenja svake instrukcije. Upravljačka jedinica (Control unit – CU) emituje sadržaj vektora maske zajedno sa svakom instrukcijom.

- Pomoću adrese PE (ne može se maskirati proizvoljan podskup PE-ova). Postoji $n=2^m$ procesnih elemenata (m-bitna maska). Svaka pozicija u maski može imati vrednost 1, 0, X. Aktivni su oni PE čija se adresa poklapa sa maskom.

- Dinamički

- Maskiranje podacima, gde maska predstavlja implicitni rezultat uslovne naredbe grananja u zavisnosti od podatka koji je lokalni za PE. Može se koristiti u kombinaciji sa drugim načinima maskiranja.

38. Nacrtati strukturu procesnog elementa kod procesiranja sa distribuiranom memorijom



- ⑩ R_{1i}, R_{2i}, R_{3i} – radni registri.
- ⑩ A_i – adresni registar.
- ⑩ I_i – indeksni registar (za adresiranje memorije zajedno sa A_i).
- ⑩ T_i – registar za komunikaciju, spregnut sa drugim T registrima preko sprežne mreže.
- ⑩ S_i – statusni registar (0 – PE pasivan, 1 – PE aktivan).
- ⑩ ADR_i – m-bitna adresa PE-a.

39. U SIMD sistemu postoji 16 procesnih elemenata. PE-ovima sa adresama 0, 1, 3, 9, 11, 15 treba dozvoliti rad, a ostalima zabraniti. Kako izgledaju maske kojima se ovo postiže ako se koristi direktno maskiranje, a kako ako se koristi adresno maskiranje?

- Direktno maskiranje:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	1	0	1	0	0	0	0	0	1	0	1	1

- ⑩ Adresno maskiranje:

0	0000	0000
1	0001	} x0x1
3	0011	
9	1001	
11	1011	
15	1111	1111

40. *Kojim sprežnim funkcijama su definisani mešanje-zamena, kub i rešetka?*

Sprežna funkcija Sf preslikava adresu X jednog PE u adresu Sf(X) drugog PE sa kojim se ostvaruje povezivanje. Kada se sprežna funkcija primeni na adresu jedne komponente sistema, dobija se adresa druge komponente sistema koja može direktno primiti podatak.

- Rešetka (dijametar $O(\sqrt{n})$, $r = \sqrt{N}$, N – broj PE)

$$M_{+1}(x) = (x + 1) \bmod N$$

$$M_{-1}(x) = (x - 1) \bmod N$$

$\vdots$

$$M_{+r}(x) = (x + r) \bmod N$$

$$M_{-r}(x) = (x - r) \bmod N$$

- Mešanje-zamena (S-E)

$$S(b_{m-1}, b_{m-2}, \dots, b_1, b_0) = b_{m-2}, \dots, b_1, b_0, b_{m-1}$$

$$E(b_{m-1}, b_{m-2}, \dots, b_1, b_0) = b_{m-1}, b_{m-2}, \dots, b_1, \overline{b_0}$$

gde su $b_{m-1} b_{m-2} \dots b_1 b_0$ – adrese PE-ova u binarnom obliku.

- Kub (dijametar $\log n$, $m = \log_2 N$ sprežnih funkcija)

$$C_i(b_{m-1}, \dots, b_i, \dots, b_1, b_0) = b_{m-1}, \dots, \overline{b_i}, \dots, b_1, b_0, \quad i = 0, 1, \dots, m - 1$$

41. *Kako se određuje zaglavlje poruke koje definiše put kroz sprežnu mrežu generalizovani kub u slučaju da se vrši emisija?*

Koristi se prosirena sema sa EX-OR zaglavljem. Broj odredista mora biti stepen dvojke (2^i na i). Odrediste adrese se moraju razlikovati na j pozicija, a preklapati na $m-j$ pozicija. Da bi se odredilo zaglavlje za slučaj emisije potrebno je poznavati informaciju o putu pre i posle grananja, i tačke grananja. Zaglavlje za slučaj emisije određeno je skupom $\{R, B\}$:

⑩ $R = r_{m-1} \dots r_1 r_0$ sadrži informaciju o putu,

⑩ $B = b_{m-1} \dots b_1 b_0$ sadrži informacije o tačkama grananja.

42. U sistemu postoji 16 procesora. Za povezivanje se koristi višestepena sprežna mreža generalizovani kub. Procesor P1 treba da pošalje poruku procesorima P4, P5, P6, P7, P12, P13, P14 i P15. Da li je emisija moguća? Zašto? Kako izgleda zaglavlje koje se dodaje poruci?

Da bi emisija bila moguća broj odredišta mora biti stepen dvojke, 2^j :

8 odredista: $j = 3$.

Odredišne adrese se moraju razlikovati na j pozicija, a poklapati na $m-j$ pozicija:

$\log_2 N = m$, $\log_2 16 = 4$, $m = 4$

□ Odredišne adrese se moraju razlikovati na 3 pozicije, a poklapati na 1 poziciji:

0100	P4
0101	P5
0110	P6
0111	P7
1100	P12
1101	P13
1110	P14
1111	P15

□ Emisija je moguća. $S \rightarrow$ izvor $S=0000$, $D \rightarrow$ odredište $D_4=0100$, $D_5=0101 \dots D_{15}=1111$

$T_i \rightarrow$ individualno zaglavlje $T_4=S \oplus D_4=0100$, $T_5=S \oplus D_5=0101 \dots T_{15}=S \oplus D_{15}=1111$

$R \rightarrow$ informacija o puta, uzima se bilo koji T_i , npr. T_4

$B \rightarrow$ informacija o tačkama grananja, $B=T_4 \oplus T_5 \oplus T_6 \oplus \dots \oplus T_{15}=0100$

Zaglavlje izgleda: $\{R, B\} = \{0100, 0100\}$

43. *Navesti karakteristike multipriocedora i multiračunara.*

• Multiprocesori (čvrsto spregnuti sistemi)

- ⑨ Multiprocesor sadrži dva ili više homogenih procesora podjednake mogućnosti.
- ⑨ Svi procesori dele pristup zajedničkoj (deljivoj) memoriji preko sprežne mreže.
- ⑨ Svaki procesor može imati malu lokalnu memoriju koja se može koristiti za pamćenje jezgra OS-a.
- ⑨ Komunikacija između procesora ostvaruje se preko deljive memorije korišćenjem zajedničkih (deljivih) promenljivih.
- ⑨ Razmene upravljačkih informacija može se ostvariti i preko procesor-procesor sprežne mreže, mada njeno postojanje nije obavezno.
- ⑨ Svi procesori dele pristup U/I kanalima i uređajima, DMA kontrolerima preko zajedničke mreže.
- ⑨ Radom celokupnog sistema upravlja jedinstveni OS koji obezbeđuje interakciju između procesora i njihovih programa na nivou posla, zadatka, skupa podataka i elementa podatka.
- ⑨ Sprežne mreže multiprocesora su dinamičkog tipa, što znači da se veze između procesora i memorija uspostavljaju po zahtevu u toku izvršenja programa.

• Multiračunari (slabo spregnuti sistemi)

- ⑨ Svaki procesor ima lokalnu memoriju i skup U/I uređaja mada mogu deliti neke periferije.
- ⑨ Procesor sa svojom memorijom i U/I uređajima zove se računarski modul ili čvor.
- ⑨ Sprega procesora sa lokalnom i memorijom i lokalnim U/I uređajima ostvaruje se preko lokalne magistrale.
- ⑨ Skup procesora može biti heterogen.
- ⑨ Kod slabo spregnutih sistema ne postoji zajednička memorija.
- ⑨ Komunikacija između procesora ostvaruje se slanjem poruka kroz sprežnu mrežu.
- ⑨ Sprega čvora sa sprežnom mrežom ostvarena preko odgovarajućeg interfejsa.
- ⑨ Sprežne mreže su statičkog tipa.

44. *Navesti politike dodele magistrale kod multiprocesorskih sistema.*

Arbitraža se može izvesti kao statička i dinamička. Kod statičke arbitraže, raspored transakcija na magistrali između potencijalnih gospodara vrši po unapred definisanom rasporedu (obično je redosled dodeljivanja kružni). Dinamička arbitraža se može podeliti po:

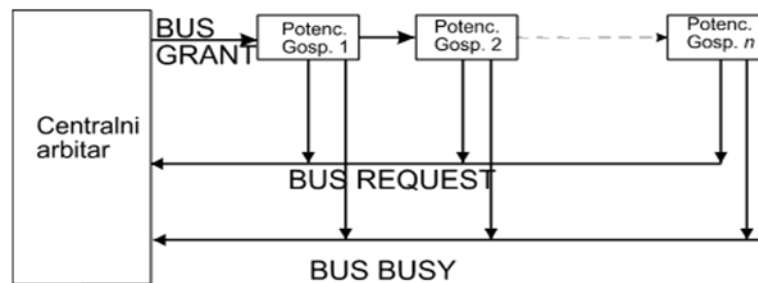
- Prioritetu – svakom potencijalnom gospodaru je dodeljen fiksni prioritet. U slučaju više zahteva magistrala se dodeljuje gospodaru sa višim prioritetom.
- Nepristrasnosti – potencijalni gospodari imaju jednak prioritet. Svakom gospodaru se koji je izdao zahtev se mora garantovati dodela magistrale pre nego što se bilo kom drugom gospodaru dodeli magistrala po drugi put. Gospodar koji je poslednji koristio magistralu dobija najniži prioritet.
- Kombinovana – politika zasnovana na prioritetu i nepristrasnosti. Obično se U/I zahtevima vrši dodela po prioritetu, a procesorskim zahtevima po principu nepristrasnosti. Multiprocesori koriste ovaj tip politike.

45. *Navesti politike oslobađanja magistrale.*

- Oslobađanje po zahtevu – gospodar magistrale ima pristup, sve dok se ne generiše drugi zahtev.
- Oslobađanje nakon obavljene transakcije – nakon obavljene transakcije gospodar oslobađa magistralu.
- Istiskivanje – gospodar koji ima viši prioritet u odnosu na tekućeg gospodara, uslovaljava da tekući gospodar oslobodi magistralu, iako nije završio sa prenosom.

46. Objasniti arbitražu na magistrali koja se zasniva na deljivim zahtevima i lančanom zahvatanju.

Svaki potencijalni gospodar magistrale izdaje zahtev za dodelu magistrale preko linije BUS REQUEST. Svi zahtevi su povezani žičanom ILI logikom.



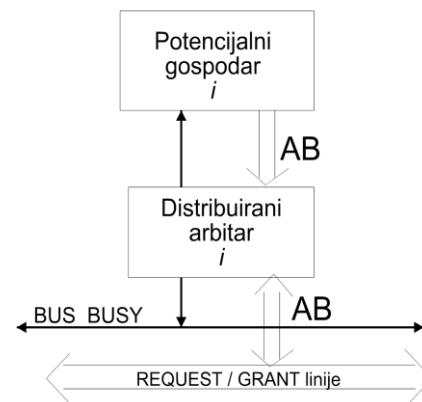
Kada **CENTRALNI ARBITAR** primi zahtev, on predaje signal BUS GRANT potencijalnom gospodaru magistrale označenim sa 1.

- Linija BUS GRANT povezuje sve potencijalne gospodare u lanac (daisy chain), tako da gospodar1 predaje signal gospodaru2, itd...
- gospodar koji je izdao zahtev ne prenosi signal dalje (prekida lanac) i aktivira liniju BUS BUSY čime ukazuje da je zahvatio magistralu.
- Kada tekući gospodar završi sa prenosom oslobađa liniju BUS BUSY i naredni ciklus arbitraže može da počne.

Ovde važi politika zasnovana na fiksnim prioritetima. Gospodar koji je bliži centralnom arbitru ima viši prioritet.

47. Objasniti distribuiranu arbitražu na magistrali

- Hardver za arbitražu je raspodeljen po potencijalnim gospodarima magistrale.
- Svaki potencijalni gospodar ima sopstveni arbitar i jedinstveni arbitracioni broj koji se koristi da razreši sudare kod jednovremenih zahteva za korišćenje magistrale.
- Kada dva ili više uređaja konkuriše za korišćenje magistrale, pobednik je onaj koji ima viši arbitracioni broj.
- Svaki potencijalni gospodar može poslati svoj arbitracioni broj (AB) na deljive request/grant linije preko svog arbitra.
- Od svih zahteva se formira zbirni arbitracioni broj, #AB.
- Nakon ovoga, svaki arbitar poredi svoj AB sa zbirnim, počev od bita najveće težine, ako je njegov broj niži od zbirnog to znači da je njegov prioritet niži i zahteva se uklanjanje.
- Na kraju na linijama ostaje arbitracioni broj onog gospodara koji ima najviši prioritet i njemu se dodeljuje magistrala.



48. *Tehnike za postizanje keš koherencije.*

Postoji više prilaza za rešavanje ovog problema.

⑩ Hardverski implementirani protokoli za postizanje keš koherencije – hardverskim tehnikama detektuju se uslovi koji mogu dovesti do nekonzistentnosti keš memorija. U ovu grupu protokola spadaju:

⑨ Snoopy keš protokoli (Njuškala)

□ Write-once

□ firefly

⑨ Direktorijumske šeme

□ Protokoli sa potpuno preslikanim adresarima

□ Protokoli sa ograničenim adresarima

□ Protokoli sa ulančanim adresarima

⑩ Softverska tehnika – Korišćenjem viših programskih jezika, korisnik deklarise podatke kao deljive (shared) ili nedeljive (local). Multiprocesorski kompajler može deklarirati podatke kao deljive ili nedeljive. Nedostatak ove tehnike je ne transparentnost multiprocesorske arhitekture za korisnika ili kompajler.

49. **Koja stanja keš bloka postoje kod write-once protokola.**

Ovaj protokol koristi W-T (write through) politiku kod ažuriranja glavne memorije pri prvoj modifikaciji keš bloka. Nakon prvog upisa keš memorija se ažurira korišćenjem W-B (write back) politike. Protokol razlikuje četiri stanja kopije keš bloka:

• **Valid** (važeća) – U kešu je kopija koja jednaka sa memorijskom kopijom i kopijama u drugim keševima (ako ih ima).

• **Invalid** (nevažeća) – Kopija je poništena.

• **Reserved** (rezervisano) – Podatak u kešu je izmenjen samo jednom i keš kopija je jednaka sa kopijom u glavnoj memoriji, koja je jedina druga (sve ostale kopije su invalidirane ako su postojale).

• **Dirty** (prljava) – Keš memorija je izmenjena više puta i kopija je jedina u sistemu (u sistemu može postojati samo jedna dirty kopija).

50. Opisati promenu stanja keš bloka u slučaju da se koristi Write-once protokol i da nastupi promašaj pri upisu

Pribavlja se kopija iz glavne memorije ili iz keša koji ima dirty kopiju. To se postiže slanjem **Read-Inv** komande koja će pribaviti i ažurirati blok i invalidirati sve ostale keš memorije. Lokalna memorija se posle ovoga nalazi u dirty stanju.

51. Koja stanja keš bloka postoje kod Firefly protokola?

- Valid-exclusive – Postoji samo jedna kopija koja je jednaka sa memorijskom kopiji.
- Shared – Postoji više kopija i sve su jednake.
- Dirty – Postoji samo jedna kopija i ona nije jednaka sa momorijskom kopijom.

Firefly koristi W-B politiku za ažuriranje privatnih blokova (blokova koji u stanju valid-exclusive i dirty), a W-T za deljive (shared) blokove.

52. Opisati promenu stanja keš bloka u slučaju da se koristi Firefly protokol i da nastupi promašaj pri upisu

Kopija dolazi iz drugih keševa ili lokalne memorije.

- Ako dolazi iz lokalne memorije stanje kopije nakon upisa je Dirty.
- U ostalim slučajevima sve kopije se ažuriraju, a novo stanje je Shared.

53. Multiprocesorski sistem se satoji od tri procesora. Svaki procesor ima svoju keš memoriju. Prikazati kako se vrši promena stanja keš blokova i glavne memorije ako se za održavanje keš koherencije koristi: Write-once protokol i Firefly protokol .Procesori pristupaju deljivoj promenljivoj X. Akcije procesora navedene su u prvoj koloni tabele. Popuniti ostale ćelije tabele.

Akcija procesora	Stanje keš kopije u P1		Stanje keš kopije u P2		Stanje keš kopije u P3		Podatak se pribavlja iz GM ili keša		Stanje kopije u glavnoj memoriji	
	a)	b)	a)	b)	a)	b)	a)	b)	a)	b)
P1 čita X	valid	valid-exclusive	invalid	/	invalid	/	GM	GM	valid	Valid
P3 čita X	valid	shared	invalid	/	valid	shared	GM	GM	valid	Valid
P3 upisuje u X	invalid	shared	invalid	shared	reserved	shared	Keš	Keš	valid	Valid
P3 upisuje u X	invalid	shared	invalid	shared	dirty	shared	Keš	Keš	invalid	Valid
P2 čita X	invalid	shared	valid	shared	valid	shared	GM	Keš	valid	valid



54. Šta su direktorijumske šeme?

Pošto je broadcast (emisiju) veoma skupo obaviti kod višestepenih mreža, komande konzistentnosti se šalju samo onim keševima koji imaju kopiju bloka. Da bi se to učinilo potrebne su tačne informacije o tome koje keš memorije imaju kopije određenog bloka. Ove informacije se smeštaju u direktorijumima (adresarima). Protokoli koji koriste informacije zapamćene u direktorijumima za održavanje koherentnosti memorijskog sistema zovu se direktorijumske šeme.

55. Kako se održava konzistentnost podataka kod direktorijumskih šema?

Svaki memorijski modul sadrži poseban direktorijum koji beleži prisustvo i stanje memorijskog bloka u određenom kešu. Svaki direktorijumski ulaz (linija) sadrži veći broj pokazivača koji ukazuju na keš koji sadrži kopiju bloka i jedan dirty bit kojim se ukazuje kada određeni keš ima ili nema, dozvolu da modifikuje određeni podatak.

56. Protokoli sa potpuno preslikanim adresarima (full mapped directories).

Svaki direktorijumski ulaz u glavnoj memoriji ima po jedan bit za svaki procesor u sistemu i jedan dirty bit. Svaki bit pridružen procesoru ukazuje na status bloka u odgovarajućem procesorskom kešu (prisutan ili ne).

- Ako je dirty bit postavljen, tada je jedan, i samo jedan, procesorski bit postavljen i taj procesor ima dozvolu da vrši upis u kešovani blok (tj. da ga modifikuje).
- Svaki keš direktorijum sadrži dva bita stanja za kešovani blok:
 - ⑨ Jedan bit ukazuje da li je blok važeći
 - ⑨ Drugi ukazuje da li se u važeći blok može vršiti upis.

57. Protokoli sa ograničenim adresarima (limited directories).

Ograničava se broj jednovremenih kopija jednog memorijskog bloka (tj. samo određeni broj procesora može sadržati kopiju bloka u svom kešu). Sličan je protokolu sa potpuno preslikanim adresarima, izuzev kada više keševa nego što je dozvoljeno zahteva kopiju jednog bloka podataka. Svaki direktorijumski ulaz u glavnoj memoriji sadrži određeni (ograničeni) broj pointera na procesore koji sadrže kopiju bloka, i jedan dirty bit koji ukazuje da li se blok može modifikovati.

58. Objasniti kako izgleda formiranje direktorijumskog lanca ako se koristi direktorijumska šema sa ulančanim adresarima.

Pretpostavimo da u sistemu ne postoji kopija bloka sa lokacijom X. Ako P1 čita lokaciju X, glavna memorija šalje kopiju bloka kešu procesora P1 i ujedno pokazivač na kraj lanca CT (Chain Termination). U adresaru glavne memorije čuva se pokazivač na procesor P1. Ako P2 zahteva čitanje lokacije X, glavna memorija šalje kopiju bloka kešu procesora P2 i pokazivač na P1. Sada se u adresaru glavne memorije čuva pokazivač na procesor P2. Ponavljanjem ovih koraka svi keševi mogu dobiti kopiju bloka. Neka jedan od procesora, recimo P2, zahteva da modifikuje sadržaj lokacije X, da bi se održala konzistentnost memorijskog sistema potrebno je da se komanda invalidacije prosledi kroz lanac. Kontroler glavne memorije ne daje dozvolu modifikacije procesoru P3 sve dok ne primi potvrdu o invalidaciji od procesora koji sadrži CT pinter.

59. Koje su operacije dozvoljene nad semaforima? Dati opis operacija u pseudo kodu

Jedine operacije, izuzev inicijalizacije, nad semaforima koje su dozvoljene **P** i **V** operacije (WAIT i SIGNAL). Nihovo dejstvo je sledeće:

```
P(S): if  $S > 0$  then
      S := S - 1
    else { blokirati proces koji je izvršio P(S)
           i smestiti ga u red čekanja pridružen semaforu S }
Izvršenje P(S) će blokirati proces ako je semafor zatvoren.
V(S): S := S + 1
      if { red čekanja pridružen semaforu S nije prazan }
      then { selektovati jedan proces iz reda čekanja i smestiti ga
            u red spremnih }
```

Izvršenje V(S) će proces koji je bio blokiran prevesti u stanje spreman.

60. Navesti razlike između semafora i monitora kao sredstava sinhronizacije i komunikacije procesa.

- Semafori – su celobrojne promenljive koje mogu uzeti samo nenegativne vrednosti. Jedine operacije, izuzev inicijalizacije, dozvoljene nad semaforima su P i V (WAIT i SIGNAL), ove operacije su nedeljive, tako da se ne može desiti da ih više procesa izvršava nad istim semaforima u isto vreme.
- Monitori – sve kritične sekcije, odnosno naredbe koje se obraćaju deljivim i promenljivama sadržane su u monitoru. Deklaracija monitora sadrži brojne procedure koje definišu operacije nad deljivim resursima (promenljivim). Opšti oblik deklaracije monitora je:

```
*deklaracija lokalnih promenljivih za monitor*
  *deklaracija procedura lokalnih za monitor*
  begin
    *inicijalizacija lokalnih promenljivih*
end ime;
```

Monitorske procedure su kritične sekcije i one mogu pristupati samo promenljivim

61. Čemu služe podaci tipa *CONDITION* kod monitora, i koje se operacije mogu izvršavati nad podacima tog tipa?

Da bi se obezbedila potrebna sinhronizacija procesa unutar monitora, deklarise se poseban tip podatka *CONDITION*. Podaci ovog tipa se mogu deklarirati samo unutar monitora. Ovi podaci se koriste kada proces želi da zakasni svoje izvršenje ili pobudi predhodno blokirani proces. VAR usl: *CONDITION*. Na ovaj način je deklarisan red čekanja usl. Dve operacije:

- **WAIT(usl)** – operacija blokira proces i dodaje ga u red čekanja pridruženom promenljivoj usl. Ova operacija automatski oslobađa pristup monitoru, tako da drugi procesi mogu da uđu u monitor.
- **SIGNAL(usl)** – omogućava da se oslobodi proces koji je bio blokirani zbog usl., ako nema ni jednog uslova koji čeka, ova naredba je bez dejstva.